

Issue-Labeler: an ALBERT-based Jira Plugin for Issue Classification

Waleed Alhindi
Department of Information
Technology
Prince Mohammad Bin
Fahd University
Khobar, Saudi Arabia
waleed_rh_hindi@hotmail.com

Abdulrahman Aleid
Department of Information
Technology
Prince Mohammad Bin
Fahd University
Khobar, Saudi Arabia
aleid.2009@gmail.com

Ilyes Jenhani
Department of Software
Engineering
Prince Mohammad Bin
Fahd University
Khobar, Saudi Arabia
ijenhani@pmu.edu.sa

Mohamed Wiem Mkaouer
Department of Software
Engineering
Rochester Institute of
Technology
Rochester, United States
mwmvse@rit.edu

Abstract— Issue labels are key drivers in software maintenance as they dictate the prioritization, organization, and ultimately the resolution of encountered issues. Consequently, mislabeling issues result in inefficient prioritization, which compromises the resolution process of these issues. Thus, to increase the accuracy and effectiveness of issue labeling in software maintenance, this paper proposes "Issue-Labeler": an automated issue labeler plugin for Jira¹, which utilizes a deep neural language model to predict an issue's type based on its title and description. Specifically, the plugin would classify an issue into three types: BUG, IMPROVEMENT, and NEW FEATURE. The issue-labeler plugin was implemented by fine-tuning Google's pre-trained ALBERT language model, using 35,889 labeled issue reports extracted from 77 projects. The plugin showed an average F1-score of 0.75, 0.58, and 0.67, respectively, for the BUG, IMPROVEMENT, and NEW FEATURE issues. The plugin will provide developers with a tool that recommends issue labels to, in turn, optimize the process of tagging and resolving these issues. Video of tool setup and runtime is available: <https://youtu.be/mi2FwaXNrR4>. Tool Webpage: <https://issue-labeler.github.io/issue-labeler-site/>. Replication package: <https://github.com/issue-labeler/>.

Keywords— Issue Classification, Jira, Software Maintenance, Plugin, ALBERT

I. INTRODUCTION

Issue tracking is a fundamental activity of software development, where developers create reports, in the form of textual descriptions, of what they anticipate as actions to perform in the project. These entries can be used to report a task, a bug, a description of a feature request, or even a question about anything related to the project. Due to the importance of tracking issues, they have been adopted by development communities. For instance, Jira is a widely adopted project management software used for tasks and workflow management, requirements and test-case management, and, pertinently to this paper, issue tracking [1]. As of 2020, Jira had been adopted by over 65,000 companies and 180,000 customers across 190 countries with 83% of Fortune 500 companies cited as utilizing some form of Jira product [2]. Since trackers receive various types of issues, developers need to label them based on their content (e.g., BUG, FEATURE), as part of their management and resolution. This labeling task can quickly become a bottleneck, especially for projects experiencing high volumes of issues, where triaging upcoming issues can be subjective, error-prone, and time-consuming [3].

As such, this project aims to provide developers with an issue-labeler Jira plugin that automatically predicts/verifies the label of a given issue. This plugin is augmented with a model that indicates the issue's type based on its title and description. The plugin may also help developers preview what is brought by the title and/or description they have used and can quickly update their 'unclear' title and/or description to point directly toward a specific issue category. The plugin also helps project triagers with the association of appropriate code reviewers. Specifically, the plugin's prediction engine is an issue-classification deep-learning language model implemented by fine-tuning Google's pre-trained ALBERT language model. Introduced by Google in 2019, ALBERT, which stands for A Lite BERT, is a smaller version of the original BERT language model [4].

ALBERT achieves a smaller size due to the factorization of BERT's embedding parameters leading to an 80% reduction in the parameters of the projection block at the expense of only a minor drop in performance [5]. While it is known that BERT and other of its variants, such as seBERT and RoBERTa, provide promising results regarding issue classification [11,12,13,16], the large size of these models becomes a challenge for their adoption in practice. That said, ALBERT still retains BERT's fundamental architecture in implementing self-supervised learning of language representations. BERT, which stands for Bidirectional Encoder Representation from Transformers, is a deep-learning language model introduced by Google in 2018 [6].

BERT incorporates a transformer-based architecture to implement the attention mechanism [7]. Attention, in this context, refers to the breakdown of inputs into smaller segments and analyzing each segment with regard to every other segment [8]. Transformers apply attention by splitting a sentence into its composite words and then analyzing each word against each other to establish the context of the sentence as a whole. Up until recently, attention was primarily implemented by utilizing recurrent neural networks (RNNs) and long-short-term-memory networks (LSTMs) [9], both sequentially analyze inputs leading to a directional limitation in applying attention techniques [6]. On the other hand, transformers are non-directional in nature, facilitating the implementation of attention techniques bi-directionally and increasing the comprehensiveness of contextually analyzing inputs [9]. Due to these capabilities, transformers have seen recent widespread use in Natural Language Processing (NLP) tasks such as text summarization, translation, and classification [6]. As such, ALBERT, A Lite BERT, had been

¹ <https://www.atlassian.com/software/jira>

used as the basis of this paper's issue-classification model by fine-tuning its parameters and training it using 35,889 labeled issue reports extracted from 77 open-source projects, including web frameworks, mobile apps and other third-party libraries [10].

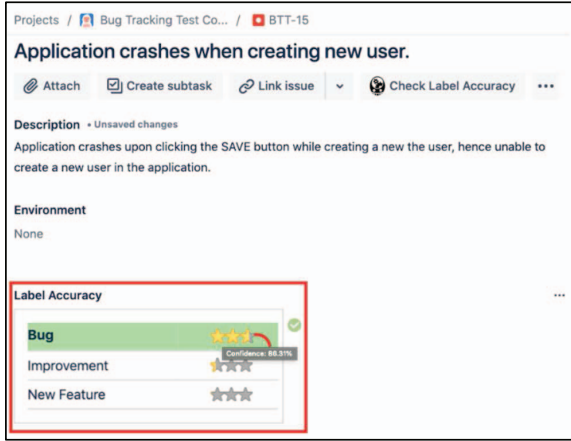


Fig. 1: Installed Issue-Labeler Tool Embedded into Jira's Native UI Displaying Prediction Label Rankings

Our issue-labeler plugin was developed to be installable into Jira due to the platform's wide reach and large-scale utilization in professional settings.

Moreover, the contributions of this paper are:

- A free, lightweight issue-classification language model, implemented by fine-tuning and training ALBERT, that classifies an issue as either a bug, improvement, or new feature based on its title and description.
- An open-source, and installable issue-labeler Jira plugin to extend the model's functionality to users and developers. Specifically, the plugin is embedded into Jira's native "Issue View" page and displays a list of the predicted labels ranked based on their corresponding confidence scores, as shown in Figure 1.
- Experimental results show that our model was able to achieve an F1-score of 0.75, 0.58, and 0.67, respectively, for the BUG, IMPROVEMENT, and NEW FEATURE Jira's categories. This score is found to be competitive with recent studies developing larger models for a different set of GitHub issue categories, namely bug, question and enhancement [11,12,13,16]. The performance can be further improved if larger models were used (e.g. BERT, RoBERTa, GPT-3, ...). For instance, with BERT, we obtained an F1-score of 0.7 across all issue categories. Our Issue-Labeler plugin uses a lite version of BERT, which can be deployed free of charge. Any larger model with better performance can be trained and deployed but with an extra cost.

II. METHOD

A. Issue Classification Model

The issue-classification model was designed by performing the following steps: (i) analyzing the SmartSHARK dataset to identify relevant data; (ii) cleaning, preprocessing, and balancing the selected data; (iii) preparing the preprocessed data for ALBERT; (iv) fine-tuning, training, and testing ALBERT. Additionally, once the issue-classification model had been finalized, a server was

constructed to host the model to facilitate the communication between the plugin and the model. The model design is shown in Figure 2.

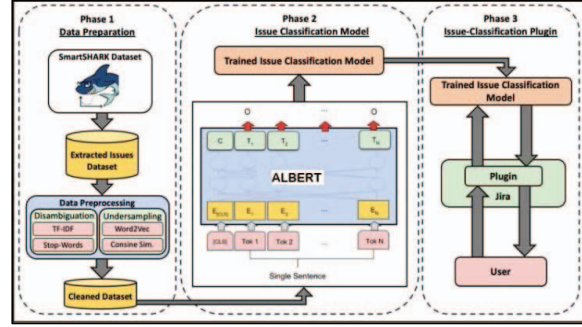


Fig. 2: Issue Classification Model Design

Issue Collection. To collect issues, we have used the SmartSHARK dataset [15]. We chose this dataset because it contains a set of 77 diverse and well-engineered projects, which also include labeled Jira's issues. To build the model's training and testing data, we initially extracted all issues, and statistically analyzed them to observe the distribution of their issue types. This analysis revealed a drastic imbalance in the representation of individual issue types. This imbalance is of concern since it would skew the model towards predicting the overly represented issue types. As such, the dataset's imbalance was addressed during the preprocessing of the data. Additionally, the target prediction classes were chosen, consisting of the following issue types: bugs, improvements, and new features. These types were chosen as they generalize across all projects [14]. As such, from an original set of around 150,000 issues, those with labels different from our target labels were filtered, resulting into 42,514 issues.

Issue Cleaning, preprocessing, and balancing. The selected data was first cleaned by removing punctuations, digits, and common English stop-words from each entry. The titles and descriptions of each entry were then concatenated such that each entry consisted of an issue type and a single string of that issue's concatenated title and description. To balance the dataset, we performed cosine similarity on the entries to identify clusters of issues with high similarity. We removed a subset of issues for each cluster of similar issues having the same issue type until the dataset became more balanced. After this process, the final dataset contains 35,889 issues distributed as follows: 12,672 bugs; 13,695 improvements; 9,522 new features.

Issue Preparation for ALBERT. The preprocessed data was then prepared for ALBERT's training/validation and testing. First, the target classes (issue types) were binarized using Scikit-Learn's one-hot-encoding function. The preprocessed data was then split into three randomized and stratified subsets: training (67.5%), validation (22.5%), and testing (10%). The entries then needed to be encoded into word embeddings using ALBERT's pre-trained tokenizer. The tokenizer splits each entry into words or word-pieces. Lastly, the data was divided into batches using PyTorch's DataLoader to distribute the GPU processing during the model's training, validation, and testing.

Training, fine-tuning and Testing the Model. ALBERT was trained and fine-tuned into the issue-classification model by training and validating ALBERT using the preprocessed training and validation datasets over a set number of epochs. By observing the training and validation losses during this process, we identified the

optimal number of epochs needed to train ALBERT without overfitting the model – which was determined to be two epochs. Next, we experimented with different setups for ALBERT by fine-tuning its hyper-parameters. The best performance was obtained with a drop-out rate = 0.1, learning rate = $1e-5$, max_length = 512 and the AdamW optimizer. The trained model's confusion matrix and performance metrics were then obtained by testing it against the testing set (see section III).

B. Issue Labeler Plugin

The Jira plugin consists of two subcomponents: a front-end, which will be placed inside Jira's native UI using Jira's Connect framework, and a Node.js backend, which serves as intermediary between the front-end and Jira's REST API as well as intermediary between the front-end and the issue-classification model.

As shown in Figure 3, the plugin functions by placing a front-end UI into Jira's native UI, fetching contextual data regarding the embedded page, such as the current project and issue. The front-end then sends this data, in the form of a JSON, to the Node.js back-end server. The Node.js server will then use the contextual data provided by the front-end to obtain more detailed information about issues currently being displayed to the user through issuing REST calls, provided by Jira's JavaScript REST API v3, to Jira's back-end data source. Namely, the Node.js back-end will retrieve that current issue's title and description using the project and issue IDs provided by the front-end. The Node.js backend will then pass these titles and descriptions to the model's server through an API. The model will then produce issue-type predictions based on this data and send it back to the Node.js server. Finally, the Node.js server will send the predictions back to the front-end so they may be displayed in a list ranking the appropriateness of each type/label based on their corresponding confidence scores.

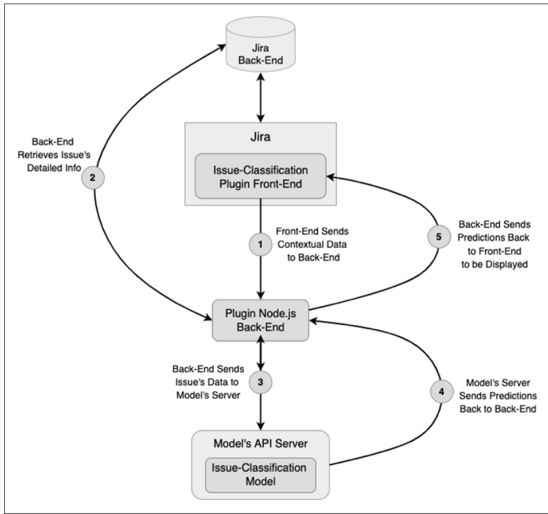


Fig. 3. High-Level Conceptual Diagram of Plugin's Operations

The issue-labeler plugin was then made installable into a Jira Cloud instance by deploying its installation package and providing users with a publicly accessible² pointing to said package. To install the plugin, this URL is pasted into a Jira

instance's "Upload App" feature found under a user's "Manage Apps" settings. Once installed, the plugin is embedded in an issue's "Issue View" page where it displays the three prediction classes (bug, improvement, and new feature) in a list ranked based on each's confidence score as shown in Figure 1.

Additionally, the stars to the right of each issue type denotes its confidence score, and an icon to the top-right of the list denotes whether the issue's current type matches the highest-confidence predicted issue type; displaying a green checkmark if they match and a red cross if they do not. Furthermore, these prediction rankings are updated "on-the-fly" to reflect any changes the user makes to that issue's title or description with a maximum duration of 0.45 seconds between user modifications and reflection of said modifications on the displayed prediction rankings.

III. PERFORMANCE EVALUATION

In this section, we assess the performance of the issue-classification model. This was done by testing the final trained model against a testing set consisting of 3,589 issues not previously seen by the model. The model was fed each of these testing issue's title and description, then the predicted issue categories were compared to the initial labels. Table 1 shows the obtained confusion matrix and Table 2 reports the model's precision, recall, and F1-score values.

TABLE 1. CONFUSION MATRIX OF THE ISSUE-CLASSIFICATION MODEL

True Labels	Predicted Labels		
	Bug	Improvement	New Feature
Bug	1027	297	63
Improvement	268	746	236
New Feature	53	271	628

TABLE 2. PRECISION, RECALL, AND F1-SCORE OF "BUG", "IMPROVEMENT", AND "NEW FEATURE" ISSUE CATEGORIES FOR THE ISSUE-CLASSIFICATION MODEL

	Precision	Recall	F1-Score
Bug	0.76	0.74	0.75
Improvement	0.57	0.60	0.58
New Feature	0.68	0.66	0.67
Micro Avg.	0.67	0.67	0.67
Macro Avg.	0.67	0.67	0.67
Weighted Avg.	0.67	0.67	0.67

Table 1 shows that the model faces the challenge of distinguishing between improvements and new features. Our qualitative analysis of false positives has revealed the challenge of learning the difference between these categories across projects, as the distinguishing features for a given category, tend to decrease as we increase the number of projects. Given the fact that this plugin is potentially used in one project setting, a trivial recommendation is to refine its model exclusively on that project's issues. This would allow

² <https://issue-labeler-plugin.herokuapp.com/install>

the model to learn each category's specificities and thus reduce ambiguity.

The performance metrics of the model, namely its precision, recall, and F1-score, were also obtained both on a per-class and overall (micro and macro) basis. As shown in Table 2, the model was quite effective in correctly predicting bugs and, to a lesser degree, new features. However, due to the aforementioned difficulties in distinguishing between new features and improvement, the model's ability to effectively predict improvements lagged behind the other two prediction classes. Moreover, the model averaged around 67% across the three prediction classes for all three performance metrics.

We initially thought of using BERT for our plugin, especially since it has been shown to provide better performances for this problem for the GitHub issue categories (bug, question and enhancement) [11,12,13,16]. However, BERT's size presented difficulties when attempting to deploy the model in the plugin. Thus, we opted to implement the issue-classification model by fine-tuning ALBERT, since it comprised a much smaller size while still offering comparable results. In fact, the BERT version of our issue-classification model averaged 70%, across the Jira's issue categories (bug, new feature and improvement), which is slightly better than our model's average of 67%. While our current plugin is shipped with an ALBERT model, developers can easily swap it with BERT, or any other variant, as long as they can eventually afford the necessary infrastructure to deploy it.

IV. CONCLUSION

In this paper, we presented Issue-Labeler, an installable Jira plugin that allows developers to more accurately and effectively label software issues during the software maintenance process. The tool performs the labeling through a deep learning-based issue classification model. The model was implemented by fine-tuning Google's pre-trained ALBERT language model and training it using issues collected from SmartSHARK repositories dataset [15]. Thus, the resultant issue-classification model predicts whether an issue is a bug, improvement, or new feature based on its title and description.

The issue-classification model scored an average of 67% across the three performance benchmarks: precision, accuracy, and recall – verifying the practical utility of the model in predicting an issue's label. The issue-labeler tool is installable into any Jira Cloud instance by simply pasting the tool package's URL, which is provided through the tool's website, into Jira's "Upload App" feature. By installing the issue-labeler tool into their Jira Cloud instance, within an issue's page, end-users would be presented with a ranking of the three prediction labels based on their corresponding confidence score. Accordingly, users would be able to label issues more accurately and, in turn, optimize their software maintenance process.

Future work will assess the tool's practical utility by obtaining direct feedback from end-users and improving upon the tool based on the feedback collected through the survey³ available on the tool's website⁴. Another direction that we foresee is making the plugin more generic by letting developers choose a different set of issue

categories from the set of categories which are available in the initial SmartSHARK dataset (e.g., epic, story, task, question, etc.) and automate the whole process of data preparation, model training/testing, and deployment.

REFERENCES

- [1] Atlassian, "What is Jira Software used for?," *Atlassian*. Available: <https://www.atlassian.com/software/jira/guides/use-cases/what-is-jira-used-for#jira-for-agile-teams>.
- [2] Atlassian, "The world's best teams work better together with Atlassian," *Atlassian*. Available: <https://www.atlassian.com/customers>.
- [3] Fan, Qiang, Yue Yu, Gang Yin, Tao Wang, and Huaimin Wang. "Where is the road for issue reports classification based on text mining?." In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pp. 121-130. IEEE, 2017.
- [4] "ALBERT: A Lite BERT for Self-Supervised Learning of Language Representations," *Google AI Blog*, 20-Dec-2019. [Online]. Available: <https://ai.googleblog.com/2019/12/albert-lite-bert-for-self-supervised.html>.
- [5] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "Albert: A lite bert for self-supervised learning of language representations," *arXiv.org*, 2020.
- [6] B. Lutkevich, "What is Bert (language model) and how does it work?," *SearchEnterpriseAI*, 27-Jan-2020. [Online]. Available: <https://searchenterpriseai.techtarget.com/definition/BERT-language-model>.
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional Transformers for language understanding," *arXiv.org*, 2019.
- [8] S. Cristina, "The transformer attention mechanism," *Machine Learning Mastery*, 27-Apr-2022. [Online]. Available: <https://machinelearningmastery.com/the-transformer-attention-mechanism/>.
- [9] K. Jose, "RNNS, lstms, CNNS, Transformers and bert," *Medium*, 12-Feb-2020. [Online]. Available: <https://medium.com/analytics-vidhya/rnns-lstms-cnns-transformers-and-bert-be003df3492b>.
- [10] F. Trautsch, S. Herbold, P. Makedonski, and J. Grabowski, "Addressing problems with replicability and validity of repository mining studies through a Smart Data Platform," *Empirical Software Engineering*, vol. 23, no. 2, pp. 1036–1083, 2017.
- [11] Siddiq, Mohammed Latif, and Joanna CS Santos. "BERT-Based GitHub Issue Report Classification." *The 1st Intl. Workshop on Natural Language-based Software Engineering*, pp. 33-36, 2022.
- [12] Trautsch, Alexander, and Steffen Herbold. "Predicting Issue Types with seBERT." *The 1st Intl. Workshop on Natural Language-based Software Engineering*, pp. 37-39, 2022.
- [13] Izadi, Maliheh. "CatIss: An Intelligent Tool for Categorizing Issues Reports using Transformers." *The 1st Intl. Workshop on Natural Language-based Software Engineering*, pp. 44-47, 2022.
- [14] Swanson, E.B.: The dimensions of maintenance. *Proceedings of the 2nd International Conference on Software Engineering, ICSE* pp. 492–497, 1976.
- [15] Trautsch, Alexander, Fabian Trautsch, and Steffen Herbold. "MSR Mining Challenge: The SmartSHARK Repository Mining Data." *arXiv preprint arXiv:2102.11540*, 2021.
- [16] Colavito, Giuseppe, Filippo Lanubile, and Nicole Novielli. "Issue report classification using pre-trained language models." *2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE)*. IEEE, 2022.

³ https://docs.google.com/forms/d/e/1FAIpQLSdwFzu-42HqChlq6vVNVrdS-FIG_GsFV-b0F3gkg0BabcFw/viewform

⁴ <https://issue-labeler.github.io/issue-labeler-site/>